

Cross-Platform Mobile Application Development

Setup, First Flutter App, GitHub, and Dart for Senior Devs

Mr. Sok Pongsametrey

Master of IT Engineering and Information System

metreysk@gmail.com · 017 856 900 · pongsametrey.com · 11 July 2026

Who's in this room today?

Show of hands — what's your background?

- Web / frontend developer pivoting to mobile
- Backend / API developer learning the client
- Bachelor's Flutter background — but inconsistent
- Career switcher / IT professional
- Government / education / NGO building software

One trait you share:

You want to leave able to ship a real mobile app — not just describe one.

Four principles. We will live by them.

01**Patterns before pace**

We re-teach the foundations through patterns so every student writes Flutter the same correct way.

02**Capstone-spine**

Every exercise builds one app — FieldLog. By W16 you have shipped it to Play Store internal testing.

03**GitHub is the workflow**

Every commit pushed by end of session. Not a submission system — a real engineering workflow.

04**AI is required**

You will use Claude / Copilot / Cursor. Disclosed. Critically reviewed. Not banned.

What you walk out with in 16 weeks

- A personal GitHub repo with 16 weeks of FieldLog exercises (E01.1 → E15.1)
- Your own copy of FieldLog: signed, published on Play Store internal testing
- A team product: a second working Flutter app, shipped with 2 classmates
- An Architecture Decision Record defending Flutter vs KMP vs native
- Fluent skills: Flutter, Dart 3, Riverpod, Drift, Dio, freezed, golden_toolkit
- Reading-level Kotlin (enough for KMP) and AI pair-programming as a real skill

Why cross-platform, why now?

Flutter

98+

active jobs in Cambodia (Indeed, 2026)

Dominant in Cambodia hiring · UI consistency · Impeller renderer · Google-backed

KMP

80%+

code-sharing rates in production KMP apps

Share business logic · keep UI native · Netflix, Cash App, McDonald's in production · JetBrains

Native

2×

engineering cost vs cross-platform

Still right for SDK-heavy apps · highest performance · two teams or two skill sets

What this course is NOT

- ✗ A Bachelor's repeat — we will not re-teach `setState`, basic widgets, or simple navigation
- ✗ A YouTube-tutorial pace — we go faster and demand patterns from week 1
- ✗ Framework-of-the-month — we go deep on Flutter and serious on KMP
- ✗ A guarantee — you will work; the course gives you the spine, you build the muscle
- ✓ **It IS: where Master's-level Flutter engineering becomes your default coding style**

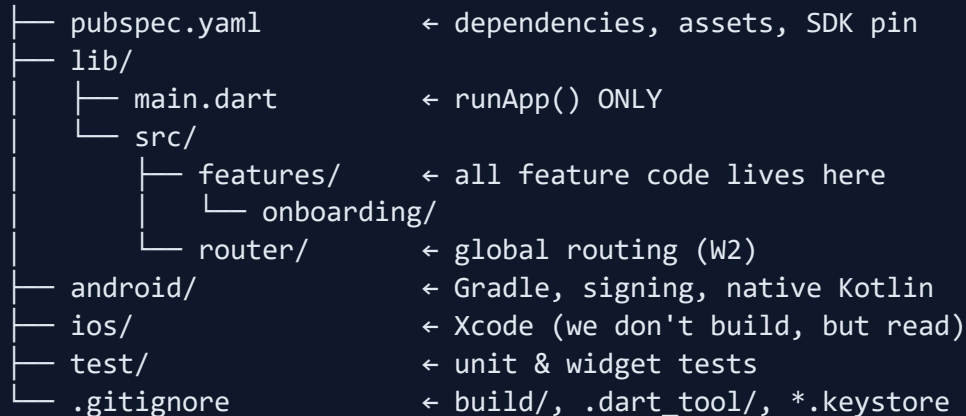
TODAY'S QUESTION

***"How do you start a Flutter project the right way,
so it survives 15 more weeks of features?"***

Most Bachelor's projects survive about 4 weeks before the architecture collapses. We fix that today.

Flutter project anatomy

fieldlog_flutter/



The single most-important rule

Source code lives under

lib/src/features/<feature>/

NEVER: lib/main.dart · lib/screens/ · lib/models/

Why: feature folders scale; technical folders fragment.

Feature folders vs technical folders

Technical folders ×

```
lib/  
├── screens/  
│   ├── profile_screen.dart  
│   └── logs_screen.dart  
├── models/  
│   ├── profile.dart  
│   └── log_entry.dart  
├── services/  
└── repositories/
```

Feature folders ✓

```
lib/src/features/  
├── profile/  
│   ├── data/  
│   ├── domain/  
│   ├── application/  
│   └── presentation/  
└── logs/  
    └── (same four)
```

main.dart — only this. Nothing else.

```
import 'package:flutter/material.dart';  
import 'src/app.dart';  
  
void main() {  
  runApp(const FieldLogApp());  
}
```

If you write business logic, theme, or routes here — you are wrong.

All app setup belongs in FieldLogApp (next slide).

FieldLogApp — your real entry point

```
class FieldLogApp extends StatelessWidget {  
  const FieldLogApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'FieldLog',  
      home: const OnboardingScreen(),  
    );  
  }  
}
```

No business logic. Just app setup, theme (W2), and the root route.

The rules of engagement

1

Visibility discipline

Every field starts private (`_field`). Make public only when outside code legitimately needs it.

2

Immutability by default

`final` on every field. `const` constructors where possible. Mutable state needs a reason.

3

Composition over inheritance

We will use `extends` exactly twice this course. Everything else: implements, mixins, or fields.

The rules of engagement, continued

4

Depend on abstractions

Never import a concrete data source from a widget or service. Always go through an abstract Repository / Service.

5

SRP test

"Can I name this class without using *and* or *or*?" If not — it does too much. Split it.

6

DIP test

"If I swap Drift for Hive tomorrow, what files change?" Only the implementation file is the right answer.

Dart 3 features you must be fluent with

Records

```
final (name, age) = ('Sok', 28);
```

Sealed classes

```
sealed class Failure {}  
class Network extends Failure {}
```

Pattern matching

```
switch (value) {  
  case (var x, var y) => x + y,  
}
```

Switch expressions

```
final label = switch (status) {  
  Loading() => 'Loading...',  
  Data(:final v) => v.toString(),  
};
```

BREAK

10 minutes — restart at the wall clock

Use this time to check that E01.1 acceptance criteria are clear in your head

Create the project

```
$ flutter create fieldlog_flutter
```

Watch the generated output. We will:

- Walk through every generated file together
- Note the bloated default lib/main.dart we will replace
- Run the app on emulator — see the default counter app working
- Then immediately impose our structure (next slide)

Impose the structure

Move generated code into proper layers:

```
$ mkdir -p lib/src/features/onboarding/presentation
$ touch lib/src/app.dart
$ touch lib/src/features/onboarding/presentation/onboarding_screen.dart

# Now rewrite main.dart to the 4-line version
# Move FieldLogApp to lib/src/app.dart
# Move OnboardingScreen to features/onboarding/presentation/
```

The first screen — OnboardingScreen

```
class OnboardingScreen extends StatelessWidget {  
  const OnboardingScreen({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () => ScaffoldMessenger.of(context)  
            .showSnackBar(const SnackBar(content: Text('Hello!'))),  
          child: const Text('Get started'),  
        ),  
      ),  
    );  
  }  
}
```

Git init, first push

```
$ git init
$ git add .gitignore # .gitignore FIRST – always
$ git commit -m "Initial .gitignore"
$ git add .
$ git commit -m "E01.1: FieldLog skeleton with feature folders"
$ git remote add origin git@github.com:<you>/fieldlog-flutter.git
$ git push -u origin main

# Commit convention: E[WW].[N]: <action>
# Always verify .gitignore caught build/, .dart_tool/
```

What you build this week

FieldLog skeleton, the right way:

- ✓ Project name: `fieldlog_flutter` (snake_case, no spaces)
- ✓ Source code under `lib/src/features/onboarding/` — NOT in `lib/main.dart`
- ✓ `main.dart` only `runApp()` — root widget in `lib/src/app.dart`
- ✓ App runs on Android emulator or physical phone
- ✓ Pushed to GitHub with proper `.gitignore` (no `build/`, no `.dart_tool/`)
- ✓ `README.md` with: app name, your name, screenshot, AI Notes section

All criteria must be met. No partial credit per exercise — effort-based grading.

If you finish E01.1 early

Custom app icon

Use `flutter_launcher_icons` to set a real icon — visible on the emulator home

Material 3 seed color

Add `ColorScheme.fromSeed` and verify the theme propagates through your one screen

Real phone deployment

Push the app to your physical phone via USB; post a photo in the class group

Stretch tasks earn public recognition, not extra grade points — but qualify you for the top mid-term band.

What FieldLog gains this week

1

Repo exists

fieldlog-flutter on GitHub; .gitignore correct; first commit pushed

2

First screen

OnboardingScreen renders 'Get started' button with Snackbar feedback

3

Structure correct

lib/src/features/<feature>/ established from day one — not refactored later

Before Week 2 begins

- Push E01.1 to GitHub — all 6 acceptance criteria met
- Install AI assistant (Claude / Copilot / Cursor); verify by asking it to explain Future<void> vs void in Dart
- Read: official Flutter — 'Navigate to a new screen and back' (15 min)
- Read: go_router README — the basics (10 min)
- Bring your laptop charged. We code live every week.

If E01.1 is not pushed by Monday, you start W2 behind. The first session won't wait for setup.